Binary Tree

Ques: Find a unique Binary Tree using the given Preorder and Inorder Traversal.

```
class Solution {
public:
    TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
        map<int, int> inMap;

        for(int i = 0; i < inorder.size(); i++) {
            inMap[inorder[i]] = i;
        }

        TreeNode* root = buildTree(preorder, 0, preorder.size() - 1,
                                   inorder, 0, inorder.size() - 1, inMap);
        return root;
    }
    TreeNode* buildTree(vector<int>& preorder, int preStart, int preEnd,
                        vector<int>& inorder,
                        int inStart, int inEnd, map<int, int> inMap) {
        if(preStart > preEnd || inStart > inEnd) return NULL;

        TreeNode* root = new TreeNode(preorder[preStart]);

        int inRoot = inMap[root->val];
        int numsLeft = inRoot - inStart;

        root->left = buildTree(preorder, preStart + 1, preStart + numsLeft,
                               inorder, inStart, inRoot - 1, inMap);
        root->right = buildTree(preorder, preStart + numsLeft + 1, preEnd,
                                inorder, inRoot + 1, inEnd, inMap);

        return root;
    }
};
```

inorder = 9, 3, 15, 20, 7

preorder = 3, 9, 20, 15, 7

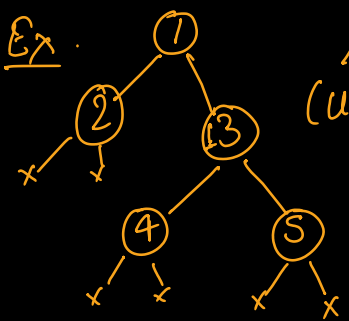
Ques: Construct a unique Binary Tree using Inorder and Postorder Traversal given.

```
public:
    TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
        if (inorder.size() != postorder.size())
            return NULL;

        map<int,int> hm;

        for (int i=0;i<inorder.size();++i)
            hm[inorder[i]] = i;

        return buildTreePostIn(inorder, 0, inorder.size()-1, postorder, 0,
                               postorder.size()-1,hm);
    }
    TreeNode* buildTreePostIn(vector<int> &inorder, int is, int ie,
                              vector<int> &postorder, int ps, int pe,
                                    map<int,int> &hm){
        if (ps>pe || is>ie) return NULL;

        TreeNode* root = new TreeNode(postorder[pe]);

        int inRoot = hm[postorder[pe]];
        int numsLeft = inRoot - is;

        root->left = buildTreePostIn(inorder, is, inRoot-1,
                                     postorder, ps, ps + numsLeft -1, hm);

        root->right = buildTreePostIn(inorder,inRoot+1, ie,
                                      postorder, ps + numsLeft, pe-1, hm);
        return root;
    }
};
```

Ques- Serialize and Deserialize a Binary Tree-

Ans- Serialize means converting a binary tree into string while deserialize means converting that string to the same binary tree.

Ex.

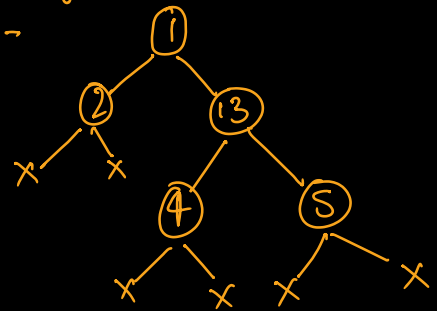


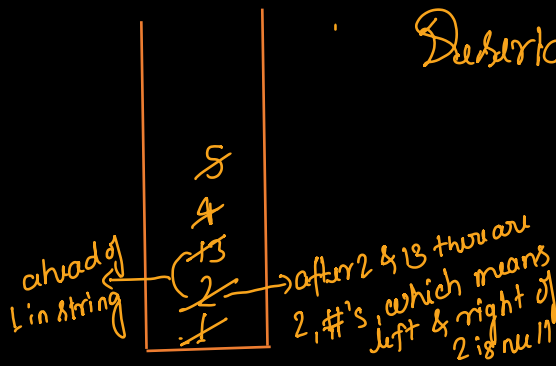
left of 2 is null, right of 2 is null

Serialize - 1, 2, 13, #, #, 4, 5, #, #, #, #, (Used level order)

Deserialize - we will take each character of this string and put it in the queue and pop that character once it's both left and right are traversed and gone in the queue.

Deserialize -





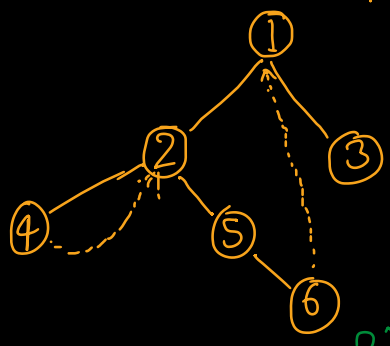
Code:

```
12
13      // Encodes a tree to a single string.
14      string serialize(TreeNode* root) {
15          if(!root) return "";
16
17          string s ="";
18          queue<TreeNode*> q;
19          q.push(root);
20          while(!q.empty()) {
21              TreeNode* curNode = q.front();
22              q.pop();
23              if(curNode==NULL) s.append("#,");
24              else s.append(to_string(curNode->val)+',');
25              if(curNode != NULL){
26                  q.push(curNode->left);
27                  q.push(curNode->right);
28              }
29          }
30          cout << s;
31          return s;
32      }
```

```
34      // Decodes your encoded data to tree.
35      TreeNode* deserialize(string data) {
36          if(data.size() == 0) return NULL;
37          stringstream s(data);
38          string str;
39          getline(s, str, ',');
40          TreeNode *root = new TreeNode(stoi(str));
41          queue<TreeNode*> q;
42          q.push(root);
43          while(!q.empty()) {
44
45              TreeNode *node = q.front();
46              q.pop();
47
48              getline(s, str, ',');
49              if(str == "#") {
50                  node->left = NULL;
51              }
52              else {
53                  TreeNode* leftNode = new TreeNode(stoi(str));
54                  node->left = leftNode;
55                  q.push(leftNode);
56              }
57
58              getline(s, str, ',');
59              if(str == "#") {
60                  node->right = NULL;
61              }
62              else {
63                  TreeNode* rightNode = new TreeNode(stoi(str));
64                  node->right = rightNode;
65                  q.push(rightNode);
66              }
67          }
68          return root;
69      }
70  };
```

Morris Traversal: → Iterative method to find inorder / postorder traversal.

It is used to find preorder / postorder traversal in O(n) time & O(1) space.



we will use threaded binary tree here.

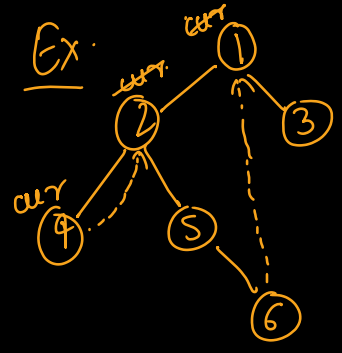
Rule of inorder –

i) If left is null, print the root and move to right.

2) Before moving to left (from root) (makethread) connect the rightmost node on the left subtree to the root then go left.

3) When we are at root and thread already exist to the rightmost node on the left subtree delete the thread and move to right.

Ex.



4, 2, 5, 6, 1, 3

→ first make the thread between 1 & 6.

→ Now move the pointer to the left as ② has left, so again move to the left. as ④ doesn't have left print it. now right of ④ is null so draw a thread between ④ & ②, as ④ is pointing to ② print ② move the cur to the right. i.e at ⑤, as it's left doesn't exist print ⑤, move to right. as left and right of ⑥ is null print ⑥, and thread is pointing to ①, so it will be printed and 'cur' will move to the right of ① → ③ then as ③ don't have left/right so it is printed.

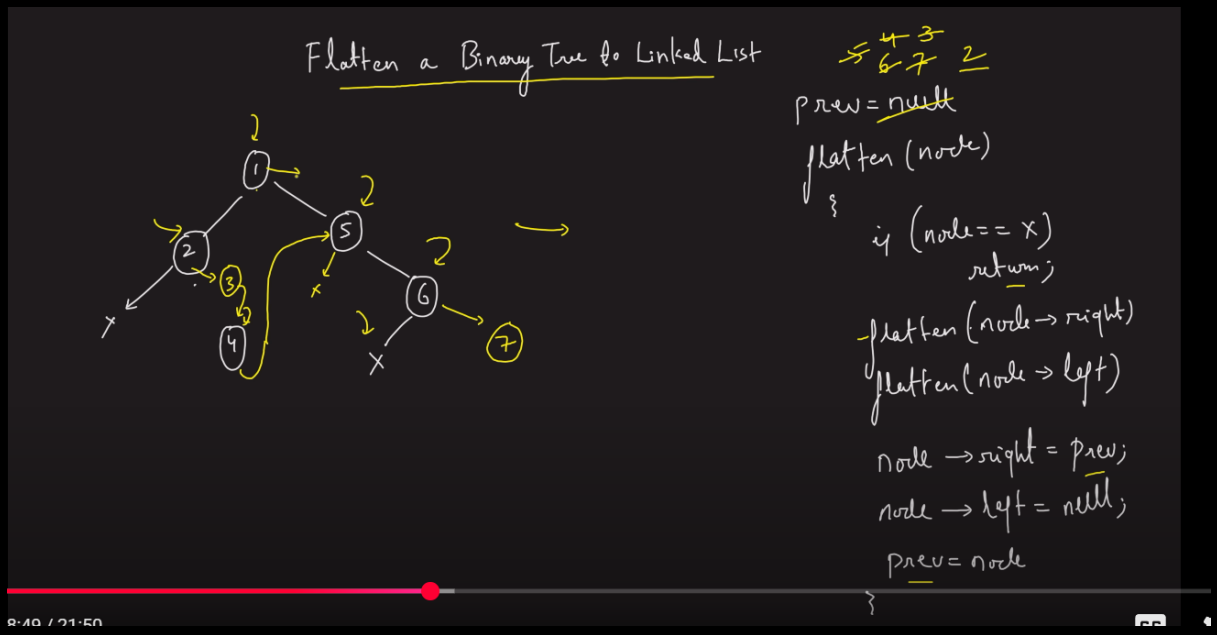
→ We have printed the node when left is null or a thread has been drawn to them.

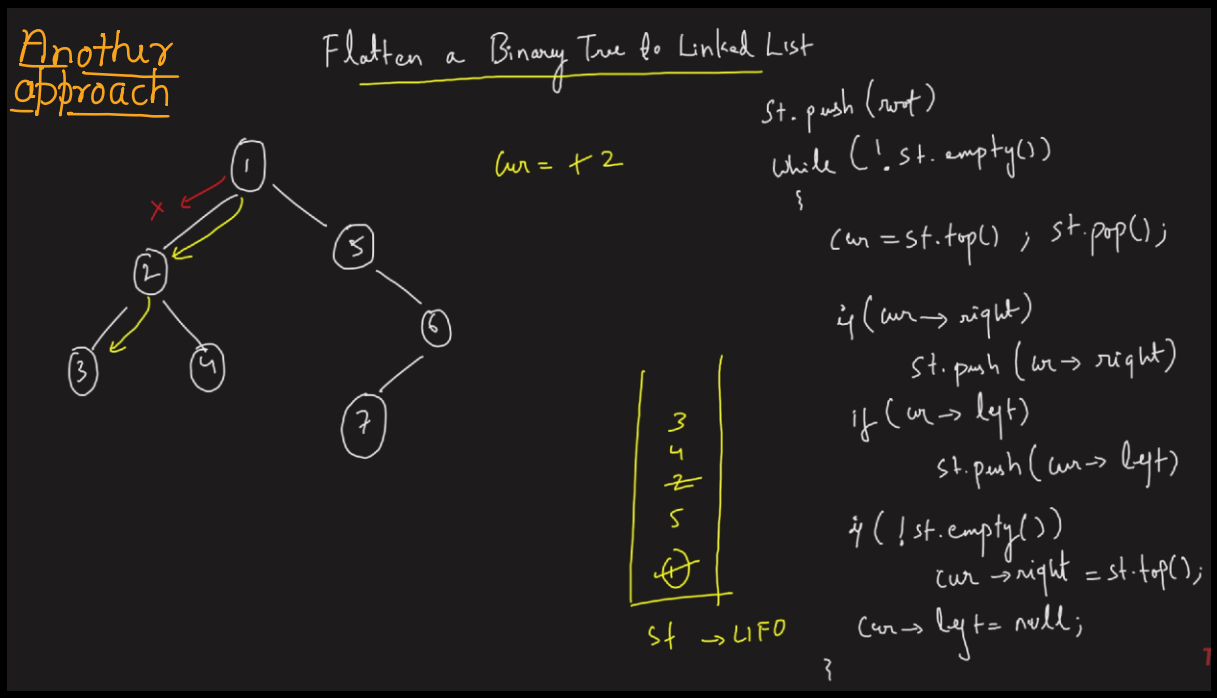

```cpp
vector<int> getInorder(TreeNode* root) {
    vector<int> inorder;
    TreeNode *cur = root;
    while(cur != NULL) {
        if(cur -> left == NULL) {
            inorder.push_back(cur->val);
            cur = cur -> right;
        }
        else {
            TreeNode *prev = cur->left;
            while(prev->right && prev->right != cur) {
                prev = prev->right;
            }

            if(prev-> right == NULL) {
                prev->right = cur;
                cur = cur->left;
            }
            else {
                prev->right = NULL;
                inorder.push_back(cur.val);
                cur = cur->right;
            }
        }
    }
    return inorder;
}
```

For Preorder:

```cpp
vector<int> getPreorder(TreeNode* root) {
    vector<int> preorder;
    TreeNode *cur = root;
    while(cur != NULL) {
        if(cur -> left == NULL) {
            preorder.push_back(cur->val);
            cur = cur -> right;
        }
        else {
            TreeNode *prev = cur->left;
            while(prev->right && prev->right != cur) {
                prev = prev->right;
            }

            if(prev-> right == NULL) {
                prev->right = cur;
                preorder.push_back(cur.val);
                cur = cur->left;
            }
            else {
                prev->right = NULL;
                cur = cur->right;
            }
        }
    }
    return inorder;
}
```

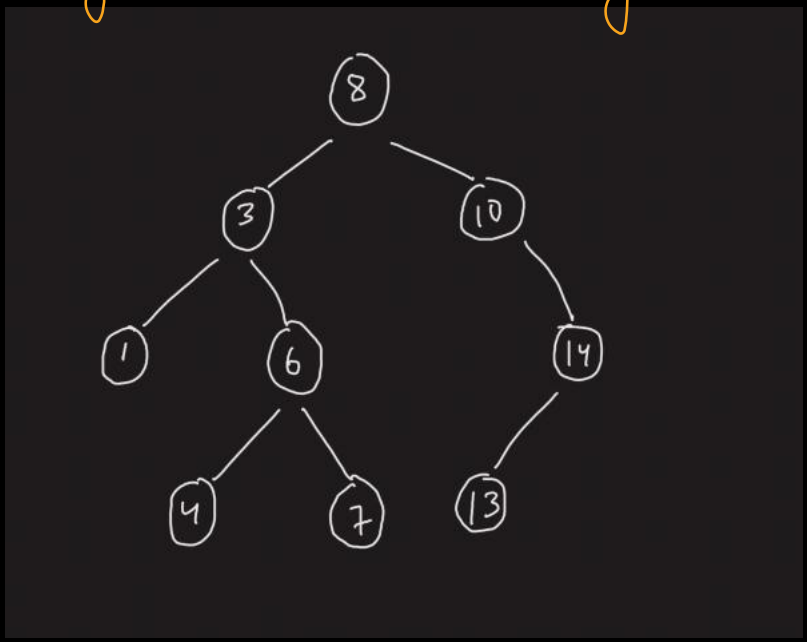
Ques: Flatten a Binary Tree;— We have to rearrange the nodes such that the are in the order as preorder traversal with right pointer pointing to the next node and left pointer to the null.





Binary Search Tree

The major difference between a binary tree and binary search tree is the left children should be smaller that the node and right children should be greater than the node. Ex-



Note :- The entire left-subtree and entire rightsub-tree should also be BST (Binary search tree) in themselves.

→ If there are dublicates then put them in the left-subtree

→ Why BST → In Binary tree all the nodes can be put in a line, which can make time complexity O(N), But in BST time complexity is generally $O(\log_2 N)$.

Search in a Binary Search Tree:

```
class Solution {
public:
    TreeNode* searchBST(TreeNode* root, int val) {
        while(root != NULL && root->val != val){
            root = val<root->val? root->left:root->right;
        }
        return root;
    }
};
```

Or-

```
TreeNode* searchBST(TreeNode* root, int val) {
    if (root == NULL) return NULL;
    if (root->val == val) return root;
    if (val < root->val) return searchBST(root->left, val);
    else return searchBST(root->right, val);
}
```

Ceil in a BST-

→ The smallest value which is greater than key.

Greater value than the given value in the BST. If greater value doesn't exist take the equal value.

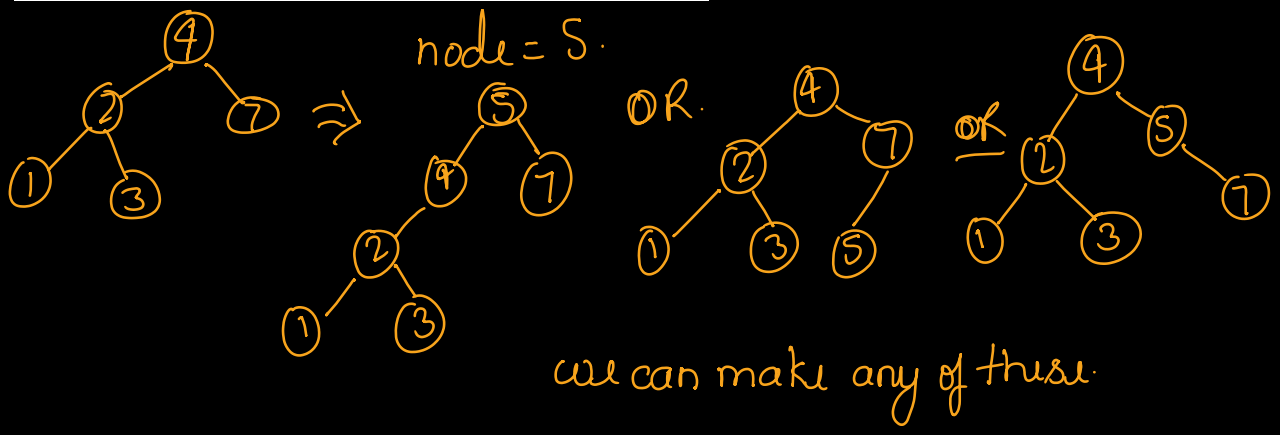
```
int findCeil(BinaryTreeNode<int> *root, int key){

    int ceil = -1;
    while (root) {

        if (root->data == key) {
            ceil = root->data;
            return ceil;
        }

        if (key > root->data) {
            root = root->right;
        }
        else {
            ceil = root->data;
            root = root->left;
        }
    }
    return ceil;
}
```

Floor In BST-

The greatest value which is smaller than given key.

```
int floorInBST(TreeNode<int> * root, int key)
{
    int floor = -1;
    while (root) {

        if (root->val == key) {
            floor = root->val;
            return floor;
        }

        if (key > root->val) {
            floor = root->val;
            root = root->right;
        }
        else {
            root = root->left;
        }
    }
    return floor;
}
```

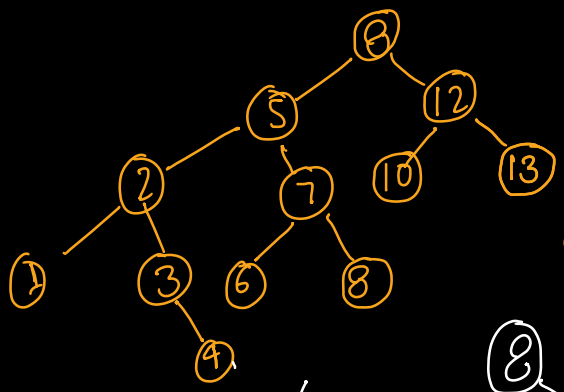
INSERT A GIVEN NODE IN A BST-



→ We will always have the chance to insert at the leaf position.

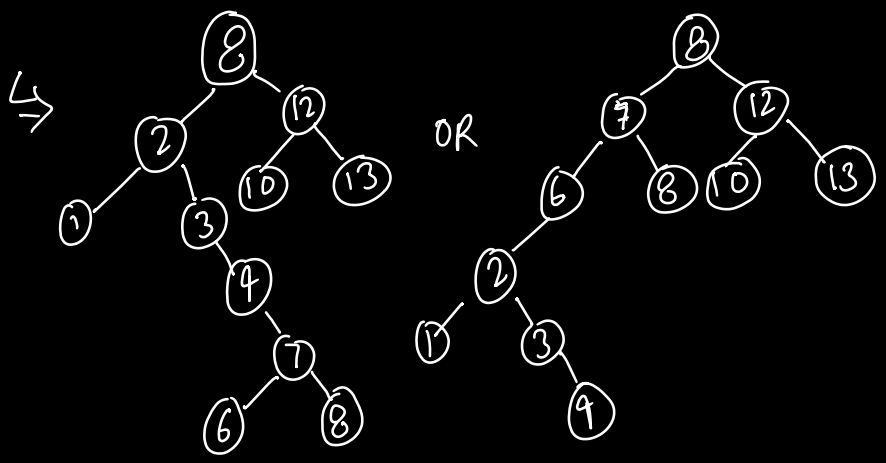
```
class Solution {
public:
    TreeNode* insertIntoBST(TreeNode* root, int val) {
        if(root == NULL) return new TreeNode(val);
        TreeNode *cur = root;
        while(true) {
            if(cur->val <= val) {
                if(cur->right != NULL) cur = cur->right;
                else {
                    cur->right = new TreeNode(val);
                    break;
                }
            } else {
                if(cur->left != NULL) cur = cur->left;
                else {
                    cur->left = new TreeNode(val);
                    break;
                }
            }
        }
        return root;
    }
};
```

Delete a node in BST-



node to be deleted = (5) Although many possible answer,

→ It is easier to connect the left/right childs of deleted node at the leaf position.



```cpp
class Solution {
public:
    TreeNode* deleteNode(TreeNode* root, int key) {
        if (root == NULL) {
            return NULL;
        }
        if (root->val == key) {
            return helper(root);
        }
        TreeNode *dummy = root;
        while (root != NULL) {
            if (root->val > key) {
                if (root->left != NULL && root->left->val == key) {
                    root->left = helper(root->left);
                    break;
                } else {
                    root = root->left;
                }
            } else {
                if (root->right != NULL && root->right->val == key) {
                    root->right = helper(root->right);
                    break;
                } else {
                    root = root->right;
                }
            }
        }
        return dummy;
    }
    TreeNode* helper(TreeNode* root) {
        if (root->left == NULL) {
            return root->right;
        }
        else if (root->right == NULL) {
            return root->left;
        }
        TreeNode* rightChild = root->right;
        TreeNode* lastRight = findLastRight(root->left);
        lastRight->right = rightChild;
        return root->left;
    }
    TreeNode* findLastRight(TreeNode* root) {
        if (root->right == NULL) {
            return root;
        }
        return findLastRight(root->right);
    }
};
```

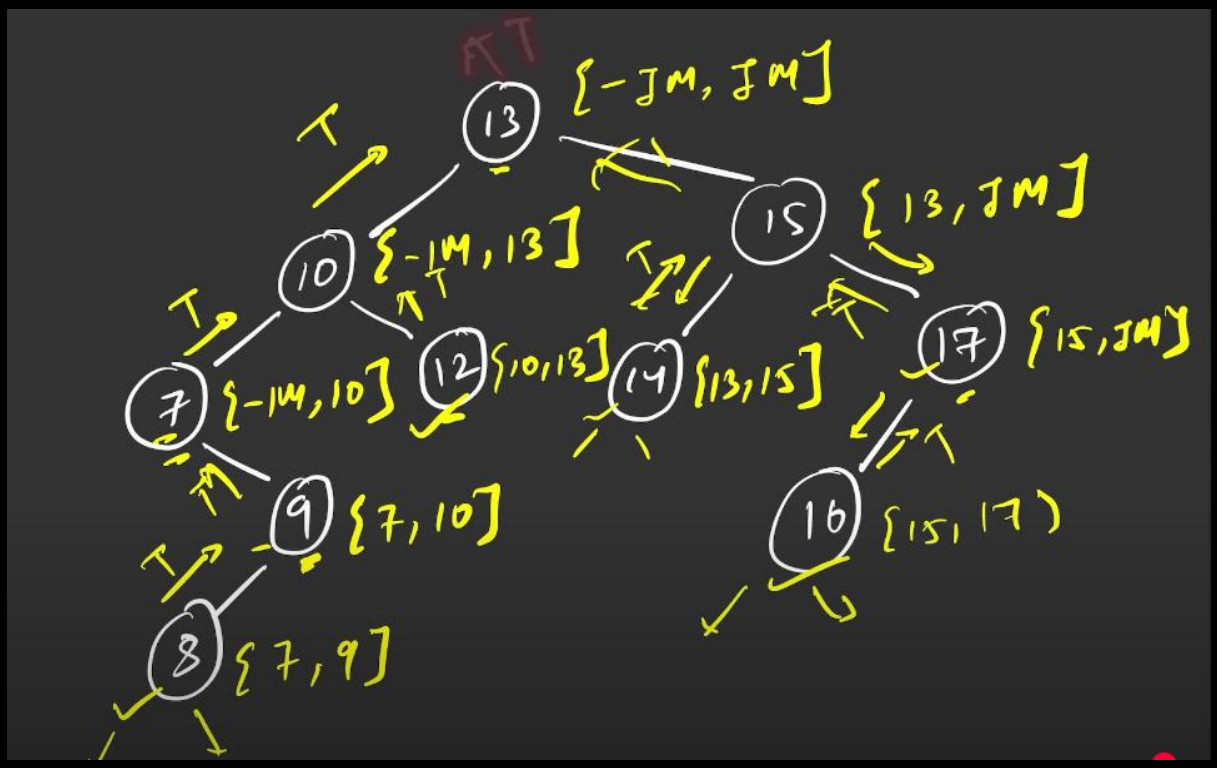
Kth smallest node in the given BST-

Approach-1- Inorder traversal in BST gives nodes in the ascending order.

Approach-2 :Don't take extra O(N) space, just keep a count and increase count when visit a node,when count==K return that node's value.

CHECK WHETHER A VALID BST OR NOT-

We will maintain a range for each node. If the value of all node is within that range, then it is valid BST.



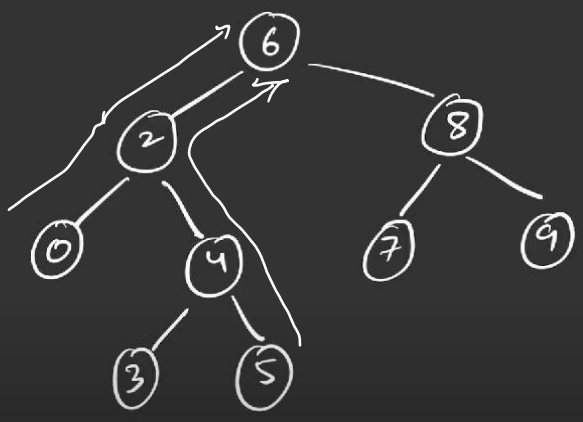

```
Solution {
public:
    bool isValidBST(TreeNode* root) {
        return isValidBST(root, LONG_MIN, LONG_MAX);
    }
```

```
private:
    bool isValidBST(TreeNode* root, long minVal, long maxVal) {
        if (root == nullptr) return true;
        if (root->val <= minVal || root->val >= maxVal) return false;
        return isValidBST(root->left, minVal, root->val) &&
               isValidBST(root->right, root->val, maxVal);
    }
};
```

LCA in BST-

↳ Lowest common Ancestor.

LCA in BST → first intersection in a path to the root.



Ex - LCA of (0, 5) → first intersection is node (2).

Note:- 1- If one node is on left and other on the right then point of split will be LCA.

2) If both the nodes are in the left (right) then one of the node will be LCA.

3) If one of the node is root then root will be the LCA.

```cpp
class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        if (root == nullptr) return nullptr;

        int curr = root->val;

        if (curr < p->val && curr < q->val) {
            return lowestCommonAncestor(root->right, p, q);
        }

        if (curr > p->val && curr > q->val) {
            return lowestCommonAncestor(root->left, p, q);
        }

        return root;
    }
};
```

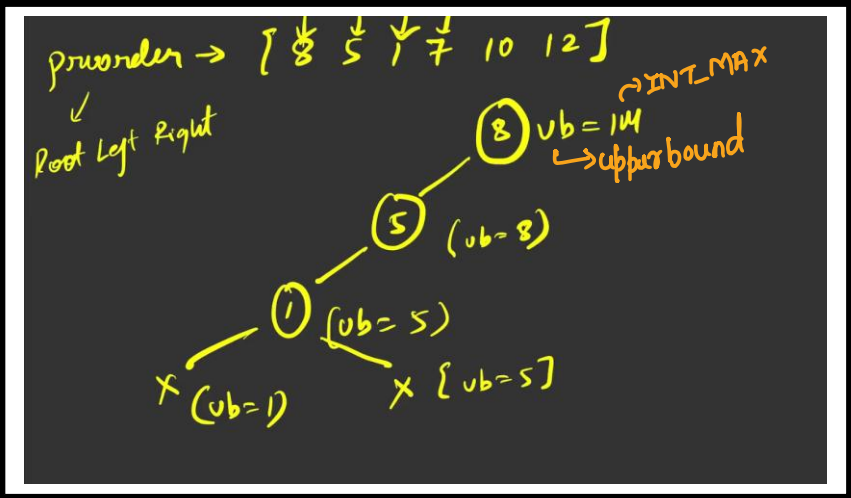
Copy Edit

→ If(p<cur && cur<q) → means point of split will be LCA.

Construct a binary search tree from a given preorder traversal-

Method-1 - If we sort the preorder, we will get inorder. Now, as we have preorder and inorder, so we can form BST.

Method-2 -

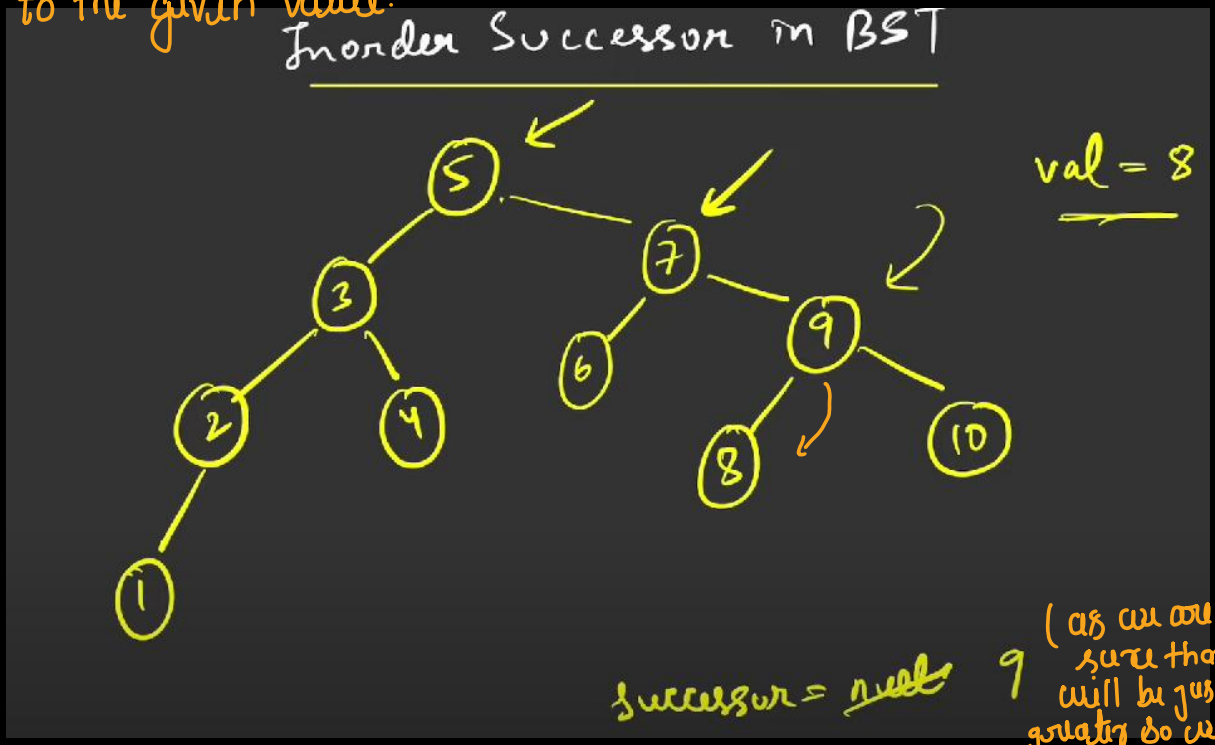


```cpp
class Solution {
public:
    TreeNode* bstFromPreorder(vector<int>& A) {
        int i = 0;
        return build(A, i, INT_MAX);
    }
Private:
    TreeNode* build(vector<int>& A, int& i, int bound) {
        if (i == A.size() || A[i] > bound) return NULL;
        TreeNode* root = new TreeNode(A[i++]);
        root->left = build(A, i, root->val);
        root->right = build(A, i, bound);
        return root;
    }
};
```

preorder array.
→ has been passed with reference, so that (pointer to the corresponding) the value changes in it in any recursive call reflect in the next call.

Inorder Successor/Predecessor in BST-

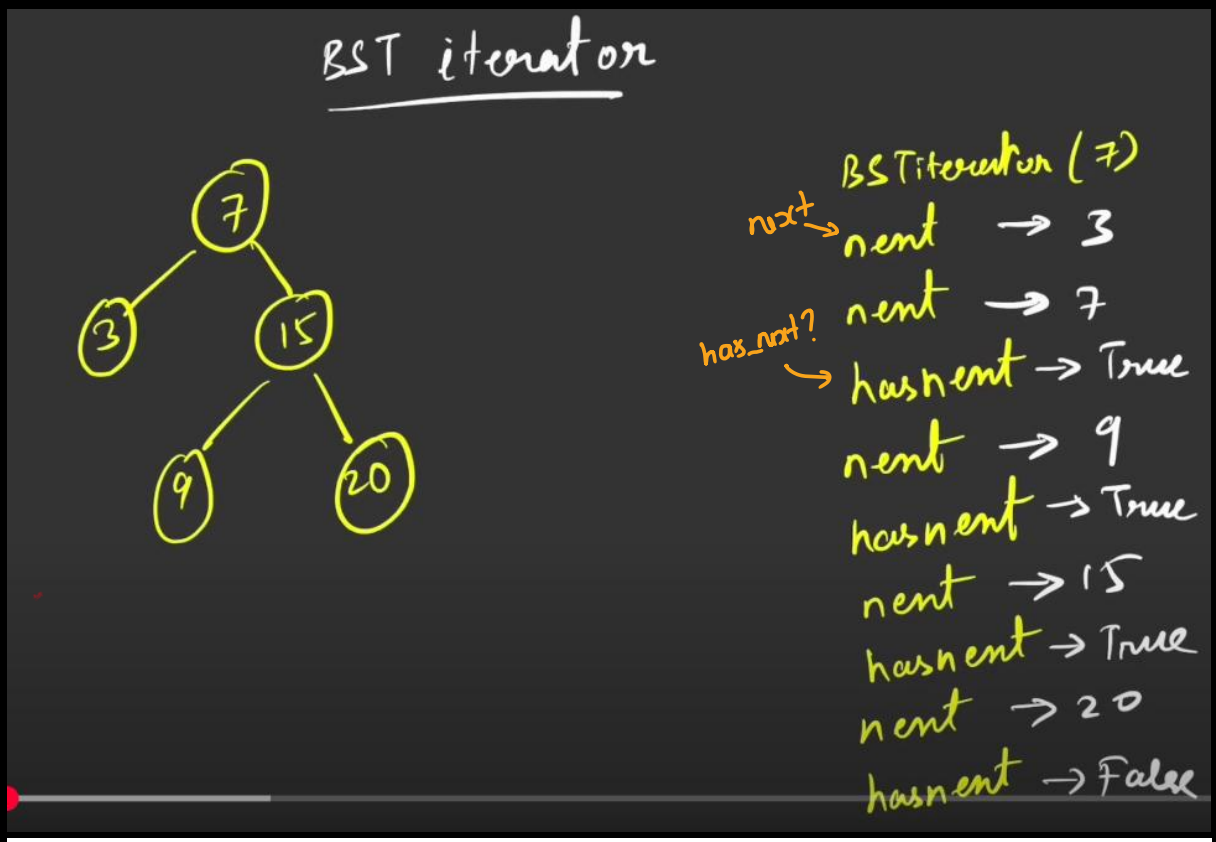
Method-1 - Store the inorder then find the next element in the inorder to the given value.



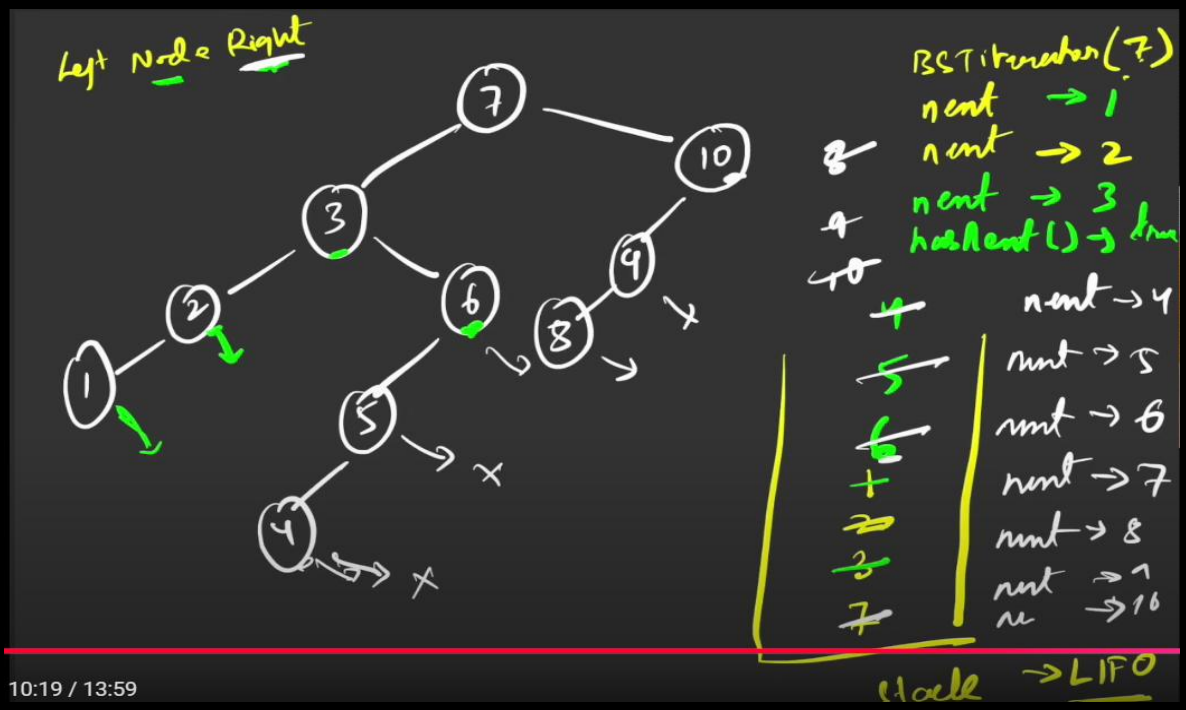
9 (as we are not sure that it will be just greater so we moved to the left).

```cpp
class Solution {
public:
    TreeNode* inorderSuccessor(TreeNode* root, TreeNode* p) {
        TreeNode* successor = NULL;

        while (root != NULL) {

            if (p->val >= root->val) {
                root = root->right;
            } else {
                successor = root;
                root = root->left;
            }
        }

        return successor;
    }
};
```

Binary Search Tree Iterator



→ Starting from the root, go to left till the last node in the iterating only left, put them in the stack, when 'next()' is called pop the top element & push the element to the right of that element and also the left elements of that pushed element.




```
12  class BSTIterator {
13      stack<TreeNode *> myStack;
14  public:
15      BSTIterator(TreeNode *root) {
16          pushAll(root);
17      }
18
19      /** @return whether we have a next smallest number */
20      bool hasNext() {
21          return !myStack.empty();
22      }
23
24      /** @return the next smallest number */
25      int next() {
26          TreeNode *tmpNode = myStack.top();
27          myStack.pop();
28          pushAll(tmpNode->right);
29          return tmpNode->val;
30      }
31
32  private:
33      void pushAll(TreeNode *node) {
34          for (; node != NULL; myStack.push(node), node = node->left);
35      }
36  };
37
```